

Lab 1 Speaking Script

CS460 Summer 2026 — ER Model & Relational Algebra · A reference, not a script to read word-for-word

Before you start: open the lab page, have Relax (dbis-uibk.github.io/relax) ready in another tab with the dataset already pasted. Take one breath. The students are here to learn, not to judge.

Opening

~1 min

- Welcome to Lab 1. Today we cover two foundational topics: the **ER model** and **relational algebra**. Both show up in every database design conversation and every query you will ever write.
- Plan for today: quick reference on ER, a modeling exercise, translating the model into SQL, then a parallel walkthrough of RA — first on paper, then live in a browser tool called Relax.
- Everything we cover is on the lab page. Open cs460-summer-2026.github.io/lab-1.html. The table of contents on the left mirrors what we'll do.

Section 1 — ER Model Quick Reference

~3 min

- The ER model is how we sketch a database before we ever write SQL. Six shapes, that's it.
- Walk through the table:
 - **Rectangle** = entity (a thing — Song, User).
 - **Ellipse** = attribute. Underline it if it's the key.
 - **Diamond** = relationship between entities.
 - **Double rectangle** = weak entity, can't exist alone.
 - **Bold line** = total participation (must be in the relationship). Single line = partial.
- Point to the first SVG diagram: *Artist records Song*. Two rectangles, a diamond in between, attributes hanging off each side. That's literally all an ER diagram is.
- Second diagram: cardinality. Arrow points to the "one" side. No arrow = "many". Memorize this.
- **Primary key**: minimal set of attributes that uniquely identifies a row.
- **Foreign key**: a column that points to another table's primary key.

Pause. Ask: "Any questions on the shapes before we use them?"

Section 2 — ER Exercise: BU Music Streaming

~5 min

- Now we model a real scenario: a music streaming service.
- Four entities — read them off the page: Songs, Artists, Users, Playlists.
- **Playlists is the weak entity.** Why? Because `playlist_id` alone is not unique. Two different users can both have a playlist with id 1. You need the owner's `user_id` to identify it. So its key is `(user_id, playlist_id)`.
- Walk the four relationships:
 1. **Records** — Artist ↔ Song, many-to-many. Why many-to-many? Songs have features; artists release many songs.
 2. **Listens** — User ↔ Song. Many-to-many. Same listener can replay the same song.
 3. **Owns** — User → Playlist. One-to-many. *Total participation on Playlist side*: every playlist must have an owner.
 4. **Contains** — Playlist ↔ Song. Many-to-many.
- Read the three discussion questions out loud. Don't answer them — let the room sit with them for a moment. If someone volunteers, great. If not, give your take after ~10 seconds:
 - Listens — partial. Some users may never play anything yet still have an account.
 - Owner deleted → playlists should be deleted (we'll see this in Rule 4).
 - Record Label could absolutely be its own entity, with a "signed to" relationship. Adds a new dimension to the model.

Section 3 — ER to Schema Translation

~6 min

- Now we go from a picture to runnable SQL. Four rules.
- **Rule 1 — Strong entity becomes a table.** Show the Songs CREATE TABLE. Every attribute is a column, key becomes PRIMARY KEY. That's it.
- **Rule 2 — Many-to-many becomes a bridge table.**
 - This is the one students always get wrong.
 - **Key insight** : you cannot store all songs by one artist as multiple columns inside that artist's row. Tables don't have variable widths.
 - So make a third table that stores the pairs. Show `Records`.

- Primary key is `(artist_id, song_id)` together. Why both? Because if it were just `song_id`, each song could only have one artist. We want features (multiple artists per song).
- Relationship attributes like `recording_date` live in this bridge table — they only make sense when you know both sides.
- **Rule 3 — Many-to-one becomes a foreign key column.** No new table, just add a column on the many side. E.g., `album_id` in Songs.
- **Rule 4 — Total participation becomes NOT NULL.** Mark the foreign key NOT NULL so every row must reference an owner. Database enforces it for you.

Section 4 — Relational Algebra Reference ~4 min

- Switch gears. Relational algebra is the math behind SQL. Every SQL query you write corresponds to an RA expression.
- Why care? Because RA forces you to think about *what* you're computing, not *how*. Query optimizers do exactly this.
- Walk the basic operators — point at the table, don't read it line by line:
 - **Selection** σ = WHERE. Filter rows.
 - **Projection** π = SELECT DISTINCT. Pick columns, kill duplicates.
 - **Cross product** $\times$ = every row of R paired with every row of S. Huge result.
 - **Set difference** $-$ = EXCEPT.
 - **Union** $\cup$ = UNION.
- Compound operators are shortcuts built from the basic five:
 - **Natural join** $\bowtie$ = cross product, then select on shared attributes. The everyday workhorse.
 - **Intersection** $\cap$, **theta-join**, **division**, **rename** ρ .
- Two facts to remember:
 1. Selection never creates duplicates (it just removes rows).
 2. Real SQL engines use bags by default — duplicates survive unless you use `DISTINCT`.

Section 5 — RA Examples (on paper) ~10 min

- Two tables: **SONGS** (15 rows, three albums) and **FEATURES** (6 rows of featured artists). Iceman, Habibti, Maid of Honour.

- We're going to walk through 7 questions. Read the question, write the RA expression on the board (or talk through it), then click "Answer" to confirm.
- **Q1.** Iceman tracks → simple selection then projection. $\pi \text{ title } (\sigma \text{ album} = \text{'Iceman'} (\text{SONGS}))$. Five songs.
- **Q2.** RnB songs → same shape, different filter. Reinforce the pattern.
- **Q3.** Songs with a featured artist → first join (natural join $\text{SONGS} \bowtie \text{FEATURES}$). Watch the row count drop from 15 to 5.
- **Q4.** Song Central Cee is on → filter Features first, then join. Show that filtering early is good practice.
- **Q5.** Features on Iceman OR Habibti → first **union** query. Two joins, then $\cup$.
- **Q6.** Features on Habibti AND Maid of Honour → **intersection**. Same shape as Q5 but with $\cap$. Answer is just Sexy Red.
- **Q7.** Songs with no featured artist → **set difference**. All song titles minus titles that appear in the join.
- Spend 1 minute on each. If pace is good, ask for student attempts before revealing the answer.

After Q7, breath. Ask: "Anyone want to see one of these run live?"

Section 6 — RelaX (live in the browser)

~12 min

- RelaX is a free in-browser RA engine. No install. dbis-uibk.github.io/relax.
- **Setup:** click the Copy button on the dataset block in the lab page. Paste into the RelaX "Group Editor". Hit "Use group". Tables show up.
- Important RelaX quirks to flag up front:
 - Type only *one* expression at a time. Multiple stacked queries crash with a parse error.
 - Can't type σ or π ? Use words: **sigma**, **pi**, **join**, **union**, **except**.
 - Set difference **-** doesn't work — use the word **except** instead. This is why the lab page uses "except" in this section.
- Walk through R1–R6, executing each one live:
 1. **R1:** $\pi \text{ title } (\sigma \text{ album} = \text{'Iceman'} (\text{Songs}))$ — verify it matches Q1.
 2. **R2:** $\pi \text{ genre } (\text{Songs})$ — 15 rows go to 4. **Projection eliminates duplicates.** This is the most important fact in this section.
 3. **R3:** build the Habibti-features query step by step. Run step 1, look at the result. Run step 2, look. Run step 3. This is what a query plan looks like.

4. **R4:** π title (Songs) except π title (Songs $\bowtie$ Features) . Songs with no features. Verify against Q7.
5. **R5:** cross product vs natural join. `Songs  $\times$  Features` gives 90 rows. `Songs  $\bowtie$  Features` gives 6. The 84 extras are nonsense pairings — Dust featuring Popcaan, etc.
6. **R6 — self join:**
- Step 1: `Features  $\bowtie$  Features` → just gives back Features. A no-op.
 - Step 2: rename both columns of one copy. No join, just renamed.
 - Step 3: rename only one column, then join. *This* is useful — pairs of artists on the same song. The pattern behind "find songs featuring two different artists."
- Reinforce: in RelaX the keyword `except` is set difference, the symbol `–` does *not* work as set difference (it's reserved for arithmetic).

Wrap-up ~2 min

- What we covered: ER shapes → ER modeling → ER to SQL → RA notation → RA in practice.
- Three things to remember:
 1. Many-to-many always becomes a bridge table. Always.
 2. Projection eliminates duplicates in pure RA. SQL doesn't, unless you say DISTINCT.
 3. Natural join requires matching column names. If two tables don't share columns, you need a bridge or a rename.
- For the assignment: redo Q1–Q7 in RelaX yourself. Try writing the bridge tables for Listens and Contains from Section 2. Submit a screenshot of any one RelaX query running.
- Office hours: [your slot]. Questions, email, post on Piazza.
- Thanks. See you next week.

If you get stuck mid-lab:

- "Great question, let me come back to that" is a complete sentence. Move on.
- If RelaX errors on a query: check that only one expression is in the editor.
- If a join returns 0 rows: the shared column names probably don't match.
- If you forget what σ does: it's WHERE.

This script is a reference — talk through the page, not the script. The lab page itself is the source of truth.
You've got this.